

Greeks Engine — Project Scope & Expectations

Project: Pipelined AAD (Algorithmic Adjoint Differentiation) Greeks computation under the Heston stochastic volatility model, in fixed-point RTL on FPGA.

Timeline: 3-month research project, target: publishable paper.

Core novelty claim: AAD computes all Greeks in one pipelined backward pass instead of n separate re-pricings. This hasn't been done in hardware before — existing FPGA-finance work accelerates pricing (via Monte Carlo) or accelerates Greeks (via bump-and-reprice), never Greeks via AAD.

1. WHAT YOU HAVE TO DO

A. Learn (the genuinely new material — finance + AAD)

Task	Concretely means	Done when
Learn options & Black-Scholes	Read the BS formula, understand what each of the 5 Greeks means economically	You can explain to someone else why Delta and Vega matter to a trading desk, without notes
Learn Heston	Understand stochastic volatility, mean-reversion, why it has no closed form	You can explain <i>why</i> Black-Scholes' constant- σ assumption breaks and what Heston changes
Learn a numerical pricing method	Study the COS/Fourier method (recommended) — deterministic, no RNG needed	You understand every arithmetic step the method performs, well enough to list them in order
Learn AAD theory	Chain rule $\rightarrow$ computational graphs $\rightarrow$ forward vs. reverse mode $\rightarrow$ tape/adjoints	You can derive the worked example ($f = xy + \sin(x)$) from memory, with different numbers
Read the literature	Smoking Adjoints, Capriotti papers, Geeraert, Danske Bank slides, Weiss/Klaisoongnoen FPGA-Heston papers, the 2024 FPGA survey, the Feb 2026 AAD-vs-FD paper	You can place your own project precisely in the gap between "AAD papers stay in software" and "FPGA papers stay with bump-and-reprice"

B. Build — software (proof of concept, before any hardware)

Task	Concretely means	Done when
Toy AAD engine	A C++/Python class overloading <code>+</code> , <code>*</code> , <code>sin</code> , etc. to auto-record a tape and compute adjoints	It reproduces the worked example ($f = xy + \sin(x)$) exactly
AAD for Black-Scholes	Extend the toy engine to the full BS pricing formula	Output Greeks match the closed-form BS Greeks formula to machine precision
AAD for Heston-COS	Extend again to the COS-method Heston pricer	Output Greeks match a trusted reference (e.g. QuantLib) within acceptable tolerance
Write the "tape schema"	A precise, ordered list of every operation the hardware will need to replicate	Detailed enough that someone else could implement the RTL from it without asking questions

C. Build — hardware

Task	Concretely means	Done when
Fixed-point bit-width budget	Decide Q-format for every intermediate value (d1, d2, exponentials, etc.), based on expected numeric range	You have a spreadsheet mapping every tape variable to a bit width, with margin for overflow
RTL forward pricing core (BS)	Hand-written Verilog implementing the BS formula as a pipeline	Simulated output price matches your software prototype, in fixed-point
RTL pipelined reverse pass (BS)	The AAD adjoint-accumulation engine, pipelined, handling the backward dependency chain	Simulated Greeks match your software AAD prototype's BS Greeks
Swap forward core to Heston-COS	Same reverse-pass architecture, new forward core	Simulated Heston Greeks match your software AAD-Heston prototype
(If time allows) synthesize to the Zynq board	Get it running on actual hardware, not just simulation	You can report real measured latency/power numbers, not just simulated ones

D. Validate & benchmark

Task	Concretely means	Done when
Correctness validation	Compare every Greek, every stage, against a trusted reference	Error is within your fixed-point precision budget, explained and bounded
Fixed-point error analysis	Quantify how much precision is lost vs. floating-point, end to end	You can state a number: "X bits of precision lost, bounded by Y"
Build a comparison baseline	A bump-and-reprice implementation (software and/or FPGA) for a fair comparison	Head-to-head numbers: latency, throughput, area, power, accuracy
Benchmark against literature	Compare your numbers to Weiss et al. and Klaisoongnoen et al.'s reported figures	You can state where you're faster/slower and why

E. Write

Task	Concretely means	Done when
Related work section	Position your work against every paper on the reading list	A reader immediately sees the specific gap you're filling
Methodology section	Describe the AAD algorithm, the RTL architecture, the fixed-point design decisions	Someone could reproduce your work from this section alone
Results section	Present correctness, precision, and benchmark data	Every claim is backed by a number you actually measured
Discussion & future work	Acknowledge Heston-Monte-Carlo-AAD as the natural next step you didn't do	Honest about scope, doesn't oversell what you built
Full draft to professor	A complete, submittable draft	You get feedback and can iterate before a real deadline

2. WHAT TO EXPECT FROM THIS PROJECT

The deliverable, in three honest tiers

Tier	What it is	Should you expect to reach it?
Must-have	Pipelined AAD Greeks engine, fixed-point RTL, Black-Scholes forward core, validated against closed-form Greeks, benchmarked against software AAD and bump-and-reprice-on-FPGA	Yes — realistically achievable in 3 months if both tracks (algorithm + hardware) run in parallel from day one
Target	Same engine with the forward core swapped to Heston-via-COS, re-validated, re-benchmarked	Plausible if the must-have tier goes smoothly and you don't lose time to hardware debugging
Stretch (future work, not a build target)	Full Heston Monte-Carlo AAD, with on-chip RNG and multi-path tape storage	No — expect to write this up as future work in your paper's conclusion, not build it

The core contribution you're expected to demonstrate

AAD computes all Greeks in one pipelined backward pass instead of n separate re-pricings, and this hasn't been done in hardware before — existing FPGA-finance work accelerates pricing (via Monte Carlo) or accelerates Greeks (via bump-and-reprice), never Greeks via AAD.

Every result you produce should ultimately support or qualify this sentence.

What "success" looks like, concretely

A working, simulated (ideally synthesized-and-running) RTL pipeline whose Greeks match a trusted software reference within a stated, bounded error.

A head-to-head comparison showing where your AAD-based approach wins over bump-and-reprice on the same FPGA — and an honest accounting of where it doesn't (e.g. area cost of the tape storage).

A fixed-point precision analysis rigorous enough to survive a reviewer asking "how do you know your numbers are trustworthy?"

A paper draft your professor is willing to submit, most plausibly to a venue in the same family as the papers you've been reading — FPGA-focused workshops/conferences like H2RC, ReConFig, or HEART, or a broader IEEE conference/journal, depending on your professor's lab's usual targets.

What you personally walk away with, regardless of exact outcome

A working AAD implementation built from first principles — a genuinely rare skill overlap (most AAD people are software-only, most FPGA people don't touch AAD).

A concrete answer to "what happens when you force a backward-dependency algorithm into a forward-flowing pipeline" — a hardware design problem that generalizes well beyond finance.

A rigorous fixed-point precision analysis under your belt, transferable across every hardware-accelerator project you do next.

A publication attempt with a clearly-stated novelty claim — not a replication of existing work on a different chip.

What's genuinely uncertain, said plainly

Whether Heston-COS (the target tier) fits in the remaining time after Black-Scholes is solid depends entirely on how smoothly the BS pipeline goes in weeks 4–5 — check in on this explicitly around week 5 rather than assuming it'll work out.

Whether your professor wants STAC-A2-style rigor (industry benchmark methodology) or a lighter custom benchmark is a conversation to have with them early — it changes how much of the "validate & benchmark" section is required vs. optional.